
B28: SQL SuperHero Program : Session 38

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

27 April 2025 at 04:52

InSteAD Of Trigger

1. **An INSTEAD OF trigger is a trigger that allows you to update data in tables via their view which cannot be modified directly through DML statements.**
2. When you issue a DML statement such as INSERT, UPDATE, or DELETE to a non-updatable view, Oracle will issue an error.
3. If the view has an INSTEAD OF trigger, it will automatically skip the DML statement and execute other DML statements instead.
4. Note that an INSTEAD OF trigger is fired for each row of the view that gets modified.
5. you can create an INSTEAD OF trigger for a view only. You cannot create an INSTEAD OF trigger for a table.

create table employees as select * from hr.employees;
create table departments as select * from hr.departments;

--> The above 2 tables are created to show the example as I do not have rights on hr schema tables.

Creating a view

```
create view vw_emp as
select employee_id, first_name, last_name, salary, d.department_id, department_name
from employees e, departments d
where e.department_id = d.department_id
```

View created.

→ Views created
from 2 table join

we can not insert in this view, bcoz it is a complex view (its made of 2 tables)

```
insert into vw_emp values (999,'vishal','jaiswal',25000,90,'Executive')
```

ORA-01779: cannot modify a column which maps to a non key-preserved table

More Details: <https://docs.oracle.com/error-help/db/ora-01779>

---> Trigger creation

```
CREATE OR REPLACE TRIGGER new_emp_trg
  INSTEAD OF INSERT ON vw_emp
  FOR EACH ROW
DECLARE
BEGIN
  -- insert a new emp first
  INSERT INTO employees(employee_id, first_name, last_name, salary, department_id, email, hire_date, job_id)
  VALUES(:NEW.employee_id, :NEW.first_name, :NEW.last_name, :NEW.salary, :NEW.department_id, :NEW.first_name,
  sysdate, 'VP');

  -- insert the department
  INSERT INTO departments(department_id, department_name)
  VALUES(l_department_id, :NEW.department_name);
END;
```

How instead of trigger



```
create or replace trigger new_emp_trg  
instead of insert on vw_emp  
for each row  
Declare
```

→ created on View

```
Begin  
    insert into employees values (:new.employee_id, :new.first_name, :new.last_name, 'vishal',  
    insert into departments values (:new.department_id, :new.DEPARTMENT_NAME, 100, 1700);  
  
End;
```



Trigger created.

Instead of 'insert on view', → insert here.

```
insert into vw_emp values (999, 'vishal', 'jaiswal', 25000, 90, 'Executive')
```

1 row(s) inserted.

→ its inserted in view

But its actually, Inserted in table.

```
select * from employees where employee_id = 999
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_P
999	vishal	jaiswal	vishalkkkkkk	535353553	04-JAN-25	vp	25000	-
999	vishal	jaiswal	vishalk	515.123.4567	04-JAN-25	vp	24500	-

2 rows selected.

→ This row inserted in table.

Another Example:

```
CREATE TABLE customer_details
```

```
(  
customer_id number(10) primary key,  
customer_name varchar2(20),  
country varchar2(20)  
);
```

```
CREATE TABLE projects_details
```

```
(  
project_id number(10) primary key,  
project_name varchar2(30),  
project_start_Date date,  
customer_id number(10) references customer_details(customer_id)  
);
```

Lets create a view based on these 2 tables:

```
CREATE OR REPLACE VIEW customer_projects_view AS  
SELECT cust.customer_id, cust.customer_name, cust.country,  
projectdtls.project_id, projectdtls.project_name,  
projectdtls.project_start_Date  
FROM customer_details cust, projects_details projectdtls  
WHERE cust.customer_id = projectdtls.customer_id;
```

Lets Create a Instead Of Trigger:

```
create or replace trigger trg_cust_proj  
instead of insert on customer_projects_view
```

Begin

```
insert into customer_details values (:new.customer_id,:new.customer_name, :new.country);  
insert into projects_details values (:new.project_id,:new.project_name, :new.project_start_Date, :new.customer_id);  
End;
```

```
create or replace trigger trg_cust_proj
instead of insert on customer_projects_view
```

instead of insert in view

insert in table

```
Begin
  insert into customer_details values (:new.customer_id,:new.customer_name, :new.country);
  insert into projects_details values (:new.project_id,:new.project_name, :new.project_start_date, :new.customer_id);
End;
```

Trigger created.

```
INSERT INTO customer_projects_view VALUES (1,'XYZ Enterprise','Japan',101,'Library management',sysdate)
```

1 row(s) inserted.

insert in view

```
select * from customer_details
```

CUSTOMER_ID	CUSTOMER_NAME	COUNTRY
1	XYZ Enterprise	Japan

But insert in table

```
select * from projects_details
```

PROJECT_ID	PROJECT_NAME	PROJECT_START_DATE	CUSTOMER_ID
101	Library management	27-APR-22	1

1 More Example

```
select department_id, count(*) total_count from employee group by department_id
```

On the above query lets create a view.

```
create view vishal as
```

```
select department_id, count(*) total_count from employee group by department_id
```

View created.

→ View on aggregate data

```
select * from vishal
```

DEPARTMENT_ID	TOTAL_COUNT
50	45
40	1
110	2
90	3

```
delete from vishal where department_id = 50
```

ORA-01732: data manipulation operation not legal on this view

→ Deletion not allowed

trigger implemented

```
2 create or replace trigger new_vishal_trg
  instead of delete on vishal → view
  for each row
  Declare
  Begin
      delete from employee where department_id = :old.department_id;
  End;
```

Trigger created.

```
delete from vishal where department_id = 50
```

→ Delete from View
(Actually from table)

1 row(s) deleted.

```
select * from employee where department_id = 50
```

no data found

→ All data
deleted
from table

Disable a single trigger

Sometimes, you may want to disable a trigger for testing and troubleshooting purposes. To disable a trigger, you use the ALTER TRIGGER DISABLE statement:

```
ALTER TRIGGER trigger_name DISABLE;
```

=====

Disable all triggers of a table

To disable all triggers associated with a table, you use the ALTER TABLE ... DISABLE ALL TRIGGERS statement:

```
ALTER TABLE table_name DISABLE ALL TRIGGERS;
```

=====

Materialized View

Materialized view in SQL is also a logical structure which is stored physically on the disc. A materialized view is a database object that contains the results of a query. Materialized views are basically used to increase query performance since it contains results of a query. They should be used for reporting instead of a table for a faster execution.

```
Create materialized view View_Name  
Build [Immediate/Deferred]  
Refresh [Fast/Complete/Force]  
on [Commit/Demand]  
as Select .....;
```

Using above syntax you can create materialized views. The Syntax includes some different optional fields:

The BUILD clause options are shown below.

IMMEDIATE : The materialized view is populated immediately.

DEFERRED : The materialized view is populated on the first requested refresh.

The following refresh types are available. **[Very Important]**

FAST : A fast refresh is attempted. If materialized view logs are not present against the source tables in advance, the creation fails.

COMPLETE : The table segment supporting the materialized view is truncated and repopulated completely using the associated query.

FORCE : A fast refresh is attempted. If one is not possible a complete refresh is performed.

The incremental refresh is commonly called FAST refresh as it usually performs faster than the complete refresh.

Fast refresh of your materialized views is usually efficient, because instead of having to recompute the entire materialized view, the changes are applied to the existing data. Thus, processing only the changes can result in a very fast refresh time.

MATERIALIZED VIEW LOG

Use the CREATE MATERIALIZED VIEW LOG statement to create a materialized view log, which is a table associated with the master table of a materialized view.

When DML changes are made to master table data, Oracle Database stores rows describing those changes in the materialized view log and then uses the materialized view log to refresh materialized views based on the master table. This process is called incremental or fast refresh. Without a materialized view log, Oracle Database must reexecute

the materialized view query to refresh the materialized view. This process is called a complete refresh. Usually, a fast refresh takes less time than a complete refresh.

Refresh on Demand:

Using this option you can add the condition for refreshing data in materialized views.

A refresh can be triggered in one of two ways.

ON COMMIT : The refresh is triggered by a committed data change in one of the dependent tables.

ON DEMAND : The refresh is initiated by a manual request or a scheduled task.

[MOST IMPORTANT AND MUST ASK]

Difference Between View And MV

Key	Views	Materialized Views
Definition	Technically View of a table is a logical virtual copy of the table created by “select query” but the result is not stored anywhere in the disk and every time we need to fire the query when we need data, so always we get updated or latest data from original tables.	On other hand Materialized views are a virtual copy of data-driven by the select result of the query will get stored in the disk.
Storage	In Views the resulting tuples of the query expression is not get storing on the disk only the query expression is stored on the disk.	On other hand in case of Materialized views query expression and resulting tuples are stored on the disk.
Query Execution	As mentioned above in case of Views the query expression is stored on the disk and not its result so query expression get executed every time when user try to fetch data from it so that user will get the latest updated value every time.	While on other hand in case of Materialized Views result of query is get stored on the disk so query expression did not get executed every time when user try to fetch the data so that user will get the latest updated value if it get change every time.
Cost Effective	As Views does not have any storage cost associated with it so they also does not have any update cost associated with it.	On other hand Materialized Views does have cost associated with it so also have update cost associated with it.
Design	Views in SQL are designed with a fixed architecture approach due to which there is an SQL standard of defining a view.	On other hand in case of Materialized Views are designed with a generic architecture approach due to which there is no SQL standard for defining it, its functionality is provided by some database as an extension.
Usage	Views are generally used when data is to be accessed infrequently and data in table get updated frequently.	On other hand Materialized Views are used when data is to be accessed frequently and data in table get updated infrequently.

updated on frequent basis.

updated on frequent basis.

Step 1:

```
CREATE table my_employees AS SELECT * from hr.employees;
```

Step 2:

-- <Before we create a materialized view log we will need to have a primary key.>

```
alter table my_employees ADD CONSTRAINT my_employees_pk PRIMARY KEY (employee_id)
```

Step 3:

-- create a materialized view log

```
CREATE MATERIALIZED VIEW LOG ON my_employees
```

Step 4:

-- Now we can create a materialized view.

```
CREATE MATERIALIZED VIEW employees_mv  
BUILD IMMEDIATE  
REFRESH FAST ON COMMIT  
AS SELECT * from my_employees;
```

Step 5:

-- Lets update the master table and change the salary of one employee.

```
update my_employees set salary = 1 where email = 'SKING';
```

Step 6:

-- Lets verify data

```
select  
*  
from MY_EMPLOYEES  
where email = 'SKING';
```

```
update my_employees set salary = 100 where employee_id = 101
```

1 row(s) updated.

Table

→ Data updated

We are practicing on livesql and it is auto commit, so commit fires automatically and because of that MV refresh.

[changes in main table → push to mv]

materialized view

```
select * from employee_mv where employee_id = 101
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	CC
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-05	AD_VP	100	-

Automatically Refreshed
(Bcoz of Commit)

We created our MV REFRESH FAST ON COMMIT [check step 4 above]. See the below Example, if we do not fire commit then changes are not coming to MV, but as soon as we do commit, it reflects in MV too.

```
declare
  l_sal number;
Begin
  update my_employees set salary = 200 where EMAIL = 'SKING';
  select salary into l_sal from my_employees where EMAIL = 'SKING';
  dbms_output.put_line(l_sal);
  select salary into l_sal from employee_mv where EMAIL = 'SKING';
  dbms_output.put_line(l_sal);
End;
```

from table we updated

Table

mv

from mv [not Refreshed, bcoz we haven't commit]

Statement processed.
200
24000

```
declare
    l_sal number;
    l_sal1 number;

Begin
    update my_employees set salary = 200 where email = 'SKING';
    select salary into l_sal from my_employees where email = 'SKING';
    dbms_output.put_line(l_sal);

    commit;

    select salary into l_sal1 from employees_mv where email = 'SKING';
    dbms_output.put_line(l_sal1);

End;
```

Statement processed.
200
200

commit fired

Data replicated bcoz mv write as Refresh on commit

Does materialized view create table?

When you create a materialized view, Oracle Database creates one internal table and at least one index. Oracle Database uses these objects to maintain the materialized view data

```
select * from user_objects where object_name = 'EMPLOYEE_MV'
```

OBJECT_NAME	SUBOBJECT_NAME	OBJECT_ID	DATA_OBJECT_ID	OBJECT_TYPE	CREATED
EMPLOYEE_MV	-	264344079	264344079	TABLE	26-APR-25
EMPLOYEE_MV	-	264344081	-	MATERIALIZED VIEW	26-APR-25

2 rows selected.

Table created automatically

same name

We created mv

You can not drop this table. you have to drop materialized view to drop table:

```
drop table mv

ORA-12083: must use DROP MATERIALIZED VIEW to drop "SQL_CQBIWZXUOVMRXBGTEPGNPYRAH"."MV"

drop materialized view mv
→ mv dropped

Statement processed.

select * from user_objects where object_name = 'MV'
→ Table is also dropped

no data found
```

Materialize View vs. Table

[Exp > 5.years]

At the segment level, the materialized view and the table are the same.

1. Improved Query Performance:

Materialized views store precomputed results of complex queries, aggregations, or joins. This can significantly improve the performance of read-heavy queries, as the results are readily available without the need to execute the entire query.

2. Reduced Database Load:

By precomputing and storing aggregations or summarized data, materialized views can reduce the load on the underlying tables during query execution. This can lead to a more efficient use of system resources and improved overall database performance.

3. Offline Data Availability:

Materialized views can be updated periodically, allowing users to query and analyze data even when the database is in a read-only or offline state. This is beneficial in scenarios where real-time data updates are not critical, and users can work with slightly stale data.

4. Enhanced Reporting and Business Intelligence:

Materialized views are often used in business intelligence and reporting scenarios where performance and response time are critical. Precomputing and storing aggregated data in materialized views allow for faster query execution, improving the responsiveness of reporting systems.

5. One of the biggest benefit of using MV is that Oracle is take care of data in sync. If we use any table instead of that, its our responsibility to take the updated data.

How to refresh all Materialized view in database?

[Exp > 7 Years]

Mainly 2 option to refresh Materialized view.

- DBMS_SYNC_REFRESH
- DBMS_MVIEW packages

The DBMS_SYNC_REFRESH package contains the APIs for synchronous refresh, a new refresh method introduced in Oracle Database 12c onwards.

The DBMS_MVIEW package contains APIs for performing refresh operations:

- **DBMS_MVIEW.REFRESH**
Refresh one or more materialized views.
- **DBMS_MVIEW.REFRESH_ALL_MVIEWS**
Refresh all materialized views.

=====

[Exp > 7 Years]

In more details explanation

In Oracle, materialized views can be refreshed in two main ways:

in-place

out-of-place

Each method has its own characteristics and use cases. Here's a detailed comparison:

In-Place Refresh

Definition: In-place refresh updates the materialized view directly. The data in the materialized view is refreshed in its existing location and the view itself remains in place during the refresh process.

How It Works:

The existing materialized view is updated with new data, replacing the old data.

This process involves applying the changes incrementally or completely (depending on the refresh method).

Types of In-Place Refresh:

Complete Refresh: Rebuilds the materialized view by executing the query that defines it.

Fast Refresh: Applies only the changes that have occurred since the last refresh, using incremental changes.

Force Refresh: Attempts to perform a fast refresh, but if it's not possible, it falls back to a complete refresh.

Advantages:

Resource Efficiency: Generally consumes fewer resources since the view itself remains in place.

Less Disk Space Usage: Does not require extra space for a new copy of the view.

Disadvantages:

Potential for Locks: Can lock the materialized view during the refresh, potentially impacting concurrent users.

Possible Performance Impact: Depending on the size of the materialized view and the complexity of the query, the refresh can affect performance.

Out-of-Place Refresh

Definition: Out-of-place refresh creates a new copy of the materialized view. The new data is loaded into a separate temporary copy of the view, which then replaces the old view once the refresh is complete.

How It Works:

A new materialized view is created with the refreshed data.

Once the refresh is complete, the new view replaces the old one atomically, making the switch seamlessly.

Advantages:

Minimal Locking: Since the new data is loaded into a separate copy, the old view remains available and unaffected during the refresh process.

No Impact on Current Users: Users continue to access the old view until the refresh is complete, avoiding disruptions.

Disadvantages:

More Disk Space: Requires additional disk space to hold the new copy of the view until the refresh is complete.

Potentially More Time-Consuming: Depending on the size of the view and the available resources, creating and replacing the view can take more time.

Can we Update Materialized View?

By default, no. Materialized views aren't updatable. See the below Steps:

table Created:

```
create table t (  
  x int primary key, y int  
);
```

Data Inserted:

```
insert into t values (1, 1);  
insert into t values (2, 2);  
commit;
```

MV Log created, because we want fast refresh:

```
create materialized view log on t
```

Lets Create the Materialized View:

```
create materialized view mv  
refresh fast  
as  
  select * from t;
```

Now letsUpdate:

```
update mv  
set y = 3;
```

```
update mv set y = 3
```

→ cannot update materialized view

ORA-01732: data manipulation operation not legal on this view ORA-06512: at "SYS.DBMS_SQL", li

More Details: <https://docs.oracle.com/error-help/db/ora-01732>

But you can use the "for update" clause to change the data in an MV:

Lets Drop and recreate the MV:

```
drop materialized view mv ;
```

```
create materialized view mv  
refresh fast for update  
as  
select * from t;
```

```
create materialized view mv refresh fast for update as select * from t
```

Statement processed.

↓
Added
'for update' clause


```
update mv set y = 3
```

```
2 row(s) updated.
```

```
insert into mv values (3,3)
```

```
1 row(s) inserted.
```

We can
do
any DML
with
materialized
view
now..

Note that the changes aren't pushed to the base table. As soon as you refresh it, the changes are lost:

```
exec dbms_mview.refresh('mv')
```

```
Statement processed.
```

→ Refreshed

```
select * from mv
```

X	Y
1	1
2	2

```
2 rows selected.
```

→ changes
lost (reverted
back)

=====
=====

can we alter materialized View

YES.

Use the ALTER MATERIALIZED VIEW statement to modify an existing materialized view in one or more of the following ways:

- To change its storage characteristic
- To change its refresh method, mode, or time
- To alter its structure so that it is a different type of materialized view

`alter table mv add z number;`

select * from mv

X	Y	Z
1	1	-
2	2	-

2 rows selected.

→ New column added in mv

select * from t

X	Y
1	1
2	2

→ table as it is

```
exec dbms_mview.refresh('MV')
```

→ refresh

Statement processed.

```
select * from mv
```

X	Y	Z
1	1	-
2	2	-

→ Data came from main table but structure remain same

Download CSV

Fast Refresh Materialized View with multiple table Joins

Yes, but every table must contain its own MV log on rowid like below:

```
create materialized view log on emp with rowid;
create materialized view log on dept with rowid;
```

And while creating MV, we must select rowid in query

```
create materialized view empdept_mv
refresh fast on commit as
select e.rowid r_rowid, d.rowid d_rowid, e.empno, e.ename, d.deptno, d.dname from emp e, dept d where e.deptno = d.deptno;
```

And make sure that They cannot have GROUP BY clauses or aggregates

=====

[Can we select Data from MV when Refresh going on. Exp > 10 Years]

When a **Materialized View (MV)** in Oracle is in **refresh status**, selecting data from it can lead to different outcomes depending on the refresh method being used:

1. Complete Refresh

- **During Refresh:** The old data in the MV is usually **deleted and replaced** with new data. If a query is executed while the refresh is ongoing:
 - You might get **inconsistent or incomplete data**.
 - If the MV is locked, the query may wait for the refresh to complete or return an error, depending on session settings and lock handling.
- **Impact:** Queries may be blocked until the refresh is complete if LOCK MODE is applied.

2. Fast Refresh

- **During Refresh:** The MV incrementally updates with changes from the source tables.
- **Query Result:** Most queries are allowed, and **data remains available** while the refresh happens.
- **Impact:** The query returns **stale data** until the refresh completes, as the new data is not fully applied yet.

3. On-Demand or Manual Refresh

- Querying depends on when the manual refresh is triggered. Data might be **outdated** if it hasn't been refreshed recently.

Query Impact Summary

Refresh Mode	Behavior
Complete Refresh	Possible delays or incomplete data; queries may wait if locking is used.
Fast Refresh	Data available but stale until refresh is complete.
Force Refresh	Behavior depends on whether it falls back to complete or fast refresh.

Recommendations

- **Use atomic refresh** to avoid incomplete data: `BEGIN DBMS_MVIEW.REFRESH('MV_NAME', 'C'); END;`
- **Enable query rewrite** if using fast refresh and query performance is critical.
- Use `DBMS_MVIEW.LAST_REFRESH_DATE` to check the last refresh time for monitoring consistency.

=====

Exp > 10 Years

What is ENABLE QUERY REWRITE in a Materialized View?

When you create a Materialized View in Oracle (or similar databases) and use `ENABLE QUERY REWRITE`, it tells the database:

"Hey! If someone writes a query that matches the data in this materialized view, you can automatically use the materialized view instead of running the slower original query."

It makes queries faster automatically, without users even knowing.

Without Query Rewrite:

- A user writes a complex query on a huge table.
- Database scans the whole big table = slow performance.

With Query Rewrite (and Materialized View):

- Database says, "Oh wait! I have a precomputed answer (materialized view)!"
- It uses that instead = very fast performance.

See the below Example:



```
CREATE MATERIALIZED VIEW sales_summary  
BUILD IMMEDIATE  
REFRESH FAST  
ENABLE QUERY REWRITE ----- See this option we used  
AS  
SELECT product_id, SUM(quantity_sold) AS total_quantity  
FROM sales  
GROUP BY product_id;
```

ENABLE QUERY REWRITE means:

Now, if someone runs a query like:

```
SELECT product_id, SUM(quantity_sold)  
FROM sales  
GROUP BY product_id;
```

Oracle will automatically use the sales_summary materialized view instead of reading the big sales table!

Result: MUCH faster query response!

Why Use ENABLE QUERY REWRITE?

- Performance boost without changing application queries
- Smart database optimization
- Useful for reporting, dashboards, OLAP systems